Lab 1 - Introducere în dezvoltarea aplicațiilor web

Echipă laborator

- **Ioan Maxim** - Master Sisteme Distribuite, Facultatea de Matematica și Informatica, Universitatea din București
    - Email: ioanmaxim2003@gmail.com / maxim.ioan@s.unibuc.ro
- **Anghel Răzvan-Alexandru** - Master Parallel and Distributed Computing Systems, Facultatea de Automatică și Calculatoare, Universitatea Politehnică din București
    - Email: (de completat ulterior)

Arhitectura standard a unei aplicații web

- Orice aplicație web modernă e bazată pe o arhitectură de tip client - server. În cadrul ei, serverul stochează datele și le poate modifica, iar clientul efectuează cererică̆tre server cu privire la afișarea sau prelucrarea acestor date.
- O aplicație web modernă urmează un flux de funcționare clar: **Utilizator -> Frontend -> API REST -> Backend -> Bază de date**.
- Fiecare componentă are un rol clar:
    - **Utilizatorul** interacționează cu interfață (click-uri, formulare)
    - **Frontend-ul** transformă acțiunile utilizatorului în cereri către server
    - **API REST** reprezintă modalitatea de comunicare între frontend și backend (transportă datele și definește operațiile permise asupra lor)
    - **Backend-ul** aplică regulile de business și interacționează cu baza de date
    - **Baza de date** asigură persistența datelor

HTTP Fundamentals

Ce este HTTP?

- Hypertext Transfer Protocol
- Protocolul prin care un client (browser, aplicație, curl) și un server își trimit date pe internet
- E caracterizat de cicluri repetate de tip cerere (request) -> procesare -> răspuns (response)
- HTTP e *stateless* - serverul nu ține minte și nu folosește cererile anterioare
    - *Cum este menținută autentificarea între paginile unui site? (cookies, sesiuni, tokens - de discutat)*

HTTPS este versiunea securizată și criptată a protocolului HTTP. Securitatea protocolului este oferită de SSL (Secure Sockets Layer) sau TLS (Transport Layer Security).

Structura unei cereri HTTP

Vom analiza cazul de mai jos ca exemplu:

```
POST /api/v1/users HTTP/1.1
Host: api.exemplu.ro
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
Accept: application/json
Content-Type: application/json; charset=utf-8
Content-Length: 78
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Connection: keep-alive

{
  "nume": "Ion Popescu",
  "email": "ion.popescu@exemplu.ro",
  "activ": true
}
```

1. Request Line

- `POST` – metoda HTTP folosită.
- `/api/v1/users` – calea resursei (URI path) către care se trimite cererea.
- `HTTP/1.1` – versiunea protocolului HTTP.

2. Request Headers

Perechi de tip `Cheie: Valoare` care trimit metadate către server:

- `Host` – domeniul serverului (obligatoriu în HTTP/1.1).
- `User-Agent` – identifică clientul (browser, script Python, Postman etc.).
- `Accept` – tipul de răspuns pe care îl așteaptă clientul (de exemplu JSON).
- `Content-Type` – formatul corpului trimis (`application/json`).
- `Content-Length` – dimensiunea corpului în octeți.
- `Authorization` – datele de autentificare (de exemplu JWT sau cheie API).

3. Linia goală (CRLF - \r\n)

Un rând gol care marchează sfârșitul antetelor și începutul corpului cererii.

4. Request Body

Datele efective transmise serverului. În cazul unui `POST` sau `PUT`, acesta conține de obicei un obiect JSON, date dintr-un formular sau un fișier încărcat. Pentru cererile simple de tip `GET`, corpul de date este de obicei

gol.

Metode HTTP principale

- GET - citește o resursă, nu trebuie să modifice nimic
- POST - creează o resursă nouă
- PUT / PATCH - actualizează o resursă existentă (integral / parțial)
- DELETE - șterge o resursă
    - *Ce înseamnă „idempotent"? De ce GET și DELETE sunt idempotențe, dar POST nu?*

Status Codes

- 2xx - succes (200 OK, 201 Created, 204 No Content)
- 4xx - eroare cauzată de client (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict)
- 5xx - eroare pe server (500 Internal Server Error)
    - *Daca trimiteți date greșit formatate către un API, ce cod v-ați aștepta să primiti?*

Exercițiul 1

Modificați cererea din exemplu astfel încât clientul să ceară de la server utilizatorul cu numele „Maria Ionescu”.

curl - testarea unui API din linia de comandă

- Utilitar care trimite cereri HTTP direct din terminal, fară interfață grafică
- Util pentru testare rapidă și debugging, chiar și fară un frontend construit
- Folosim `https://jsonplaceholder.typicode.com` - API REST public, gratuit, care simulează un backend real (fară să salveze efectiv datele)

Exercițiul 2

Rulați comenzile de mai jos care folosesc utilitarul `curl`:

```
curl https://jsonplaceholder.typicode.com/posts/1
```

```
curl -v https://jsonplaceholder.typicode.com/posts/1
```

```
curl -X POST https://jsonplaceholder.typicode.com/posts -H "Content-Type:
application/json" -d '{"title": "test", "userId": 1}'
```

```
curl -X DELETE https://jsonplaceholder.typicode.com/posts/1
```

Opțiuni curl folosite mai sus

- `-v` (verbose) - afișează tot schimbul: conexiunea, headerele cererii (`>`) și cele ale răspunsului (`<`); util pentru debugging
- `-X` - specifică explicit metoda HTTP (GET e implicită dacă lipsește; dacă folosiți `-d`, curl trece automat pe POST chiar și fară `-X`)
- `-H "Header: valoare"` - adaugă un header custom cererii (poate fi folosit de mai multe ori pentru mai multe headere)
- `-d '...'` - trimite date în body-ul cererii; poate fi text simplu, JSON, sau format `cheie=valoare`
- *Ce se întâmplă daca omiteți headerul Content-Type la POST?*
- **Terminal recomandat:** Git Bash, WSL pe Windows sau orice terminal pe MacOS/Linux. Pe Windows nu folosiți PowerShell sau CMD pentru aceste comenzi deoarece nu vor funcționa.

JSON

- JSON - formatul standard prin care API-urile web trimit și primesc date structurate
- Nu e un protocol - e doar o convenție de formatare a textului; HTTP e protocolul care îl transportă
- Tipuri de bază: string, number, boolean, null, object, array

Exemplu - răspunsul unui GET:

```
{
  "userId": 1,
  "id": 1,
  "title": "...",
  "body": "..."
}
```

- Cum ați reprezenta soldul unui cont bancar în JSON?

JavaScript

- **JavaScript** este limbajul de programare folosit pentru a crea interacțiuni în browser și pentru logica aplicațiilor web.
- **Node.js** este un runtime JavaScript care permite rularea codului JS și pe un computer, nu doar în browser.

Exercițiul 3

1. Mergeți la https://nodejs.org/en/download și descărcați ultima versiune stabilă de Node.js pentru sistemul vostru de operare.
2. Selectați `npm` (Node Package Manager) și `nvm` (Node Version Manager).
3. Rulați comanda `node -v` pentru a verifica că Node.js s-a instalat corect.

- `fetch()` e modul standard prin care o pagină web trimite cereri HTTP din browser, fără reîncărcarea paginii.
- Practic, este echivalentul lui `curl`, dar rulat în interiorul unei pagini web / al unui fișier JS.

Exercițiul 4

- Deschideți VSCode (sau alt editor preferat)
- Descarcați fișierul lab1.js și folderul în care acesta se află
- Deschideți un terminal în același folder și rulați: `node lab1.js`
- Observați răspunsul afișat în terminal

De reținut

- `fetch` - cere și așteaptă răspunsul de la server
- `JSON.stringify()` transformă un obiect JS în text JSON (pentru body)
- `response.json()` efectuează operația inversă - parsează automat body-ul JSON primit înapoi într-un obiect JS
    - *Puteți identifica, în fișierul primit, echivalentul fiecărei părți din comenzile curl scrise anterior?*